

Support Vector Machines (SVM)

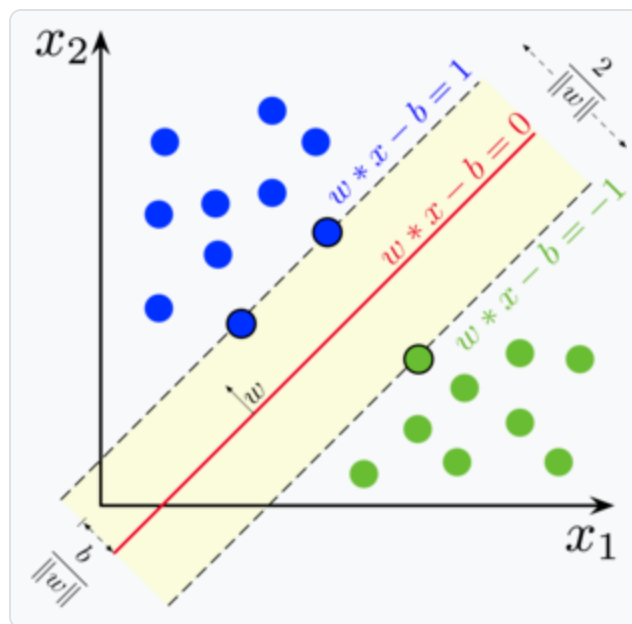
1. The Intuition & Math: Maximum Margin

SVM tries to find the "widest street" (margin) that separates the two classes.

The Concept

Imagine the decision boundary as a line (or hyperplane). Many lines can separate the data. SVM chooses the one that is farthest from the nearest data points of both classes.

- **Support Vectors:** The data points closest to the boundary. These are the "pillars" that hold up the street.
- **Margin:** The distance between the boundary and the support vectors.



SVM finds the optimal hyperplane (WikiCommons)

The Math

We want to find weights w and bias b to define our boundary $w^\top x + b = 0$.

Hard-margin SVM (Perfectly Separable)

If the data can be perfectly separated, we minimize the norm of w (which maximizes the margin $\frac{2}{\|w\|}$):

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 \quad \text{s.t. } y_i(w^\top x_i + b) \geq 1.$$

Soft-margin SVM (Handling Errors)

Real data is messy. We allow some points to violate the margin by adding "slack" variables ξ_i . We control the trade-off with parameter C :

$$\min_{w,b,\xi} \frac{1}{2} \|w\|_2^2 + C \sum_i \xi_i, \quad y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

C is crucial:

- **High C:** Strict. Tries hard not to miss any point. Risk of overfitting.
- **Low C:** Loose. Allows more errors for a wider margin. Smoother boundary.

🎓 Student Analogy: The "Strictness" Parameter

- **High C = "Strict Teacher":**
 "I will not tolerate ANY mistakes!"
 Result: The model contorts itself to classify every single training point correctly.
 Risk: Overfitting (Hard Margin).
- **Low C = "Chill Teacher":**
 "It's okay to miss a few if the general rule is good."
 Result: The model accepts some errors to keep the decision boundary simple and wide.
 Benefit: Better Generalization (Soft Margin).

2. Convex Optimization: Why SVMs are Reliable

One reason SVMs are so popular is that they are **Convex**. But what does that mean?

The "Skate Bowl" Analogy

Imagine a skate park with a single, smooth, giant bowl. If you drop a marble anywhere in the bowl, gravity will eventually pull it to the exact same spot at the very bottom. This is **Convexity**.

- **No "Traps" (Local Minima):** Unlike a rugged hiking trail where you might get stuck in a small pothole and think you're at the bottom, a convex shape has only one bottom. You

can't get lost.

- **Student Takeaway (Reliability):** If you and your classmate train an SVM on the same data, you will get the **exact same model**. There is no "luck" involved in finding the answer!



Why it matters: Unlike Neural Networks (which are non-convex and can get stuck), SVMs are very stable and predictable. If you train an SVM twice on the same data, you'll get the same result!

3. Visualization: The "Street"

Below is a visual representation of how SVM defines the margin.

Key Insight: Only the Support Vectors (highlighted) matter! You can move the other points around, and as long as they stay outside the margin, the decision boundary won't change.

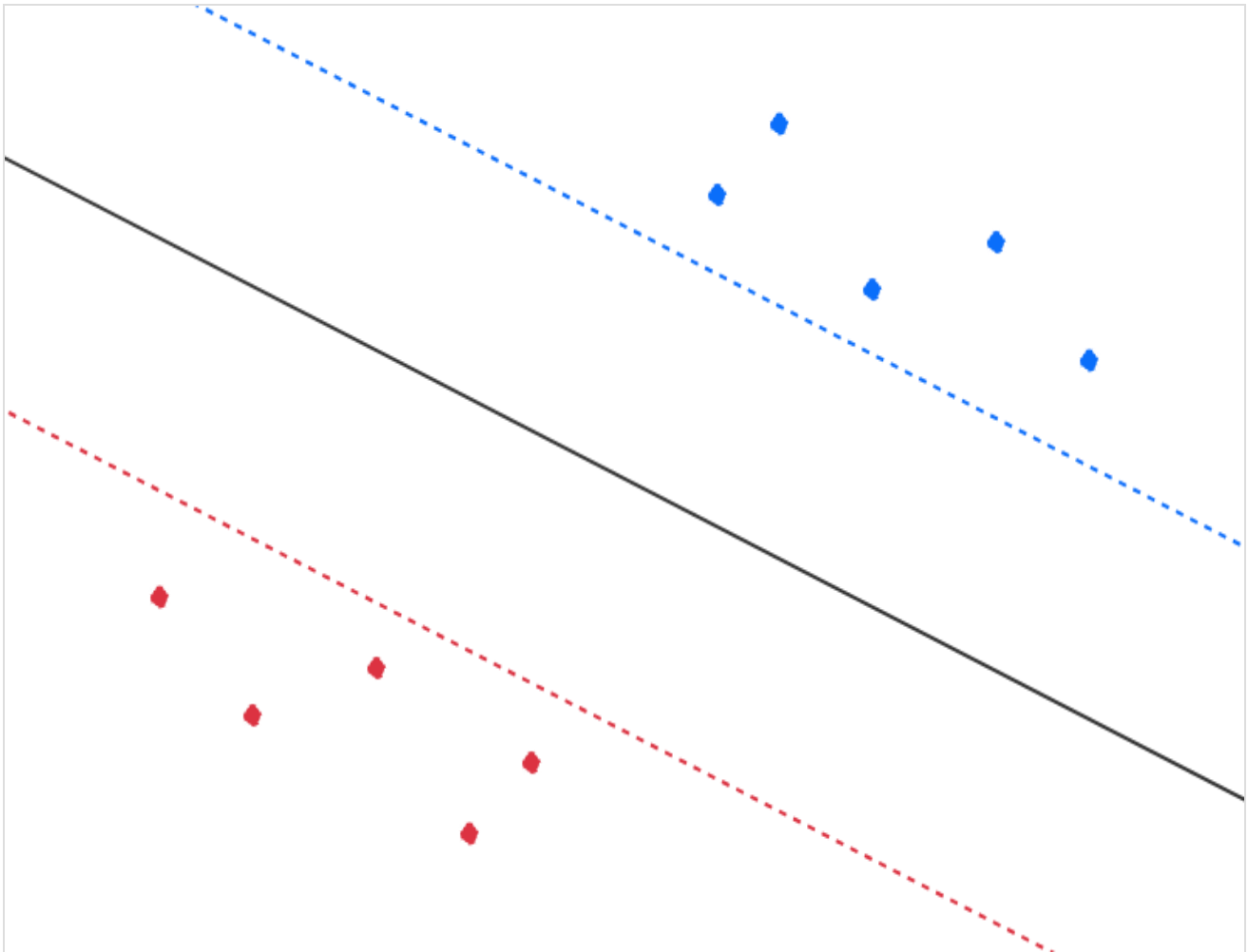
Proceed to the interactive demo to build your own SVM.

4. Interactive Demo: Build Your Own SVM

Click on the canvas to add points. The SVM will retrain automatically (using SGD).

C Parameter (Regularization):

Current C: 0.01



Note: This runs a Javascript-based Gradient Descent solver (Pegasos Algorithm) in the browser. It approximates the optimal hyperplane.

5. Python Implementation

Using **SVC** (Support Vector Classifier) from scikit-learn.

```
from sklearn.svm import SVC
from sklearn.datasets import make_blobs
import numpy as np
```

```

import matplotlib.pyplot as plt

def svm_demo():
    # 1. Generate Data
    X, y = make_blobs(n_samples=100, centers=2, random_state=6)

    # 2. Train SVM (Linear Kernel)
    clf = SVC(kernel='linear', C=1.0)
    clf.fit(X, y)

    # 3. Visualization
    plt.figure(figsize=(8, 6))
    plt.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=plt.cm.Paired)

    # Plot the decision function
    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    # Create grid to evaluate model
    xx = np.linspace(xlim[0], xlim[1], 30)
    yy = np.linspace(ylim[0], ylim[1], 30)
    YY, XX = np.meshgrid(yy, xx)
    xy = np.vstack([XX.ravel(), YY.ravel()]).T
    Z = clf.decision_function(xy).reshape(XX.shape)

    # Plot decision boundary and margins
    ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,
               linestyles=['--', '-', '--'])

    # Plot support vectors
    ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
               s=100,
               linewidth=1, facecolors='none', edgecolors='k',
               label='Support Vectors')

    plt.title(f"SVM Decision Boundary (C=1.0)\nAcc: {clf.score(X, y):.2f}")
    plt.legend(loc="upper right")
    plt.show()

```

```
svm_demo()
```

6. Step-by-Step Math: Solving for w and b

Let's solve a simple "Hard Margin" SVM problem manually to see exactly how we find w and b . No coding, just algebra!

The Dataset

We have 4 points in 2D space:

- **Class -1 (Red):** $A(1, 2)$ and $B(2, 1)$
- **Class +1 (Blue):** $C(5, 6)$ and $D(6, 5)$

Our goal is to find the hyperplane $w \cdot x + b = 0$ that separates them with the maximum margin.

Step 1: Identify Support Vectors

By visual inspection (or checking distances), the points closest to the gap between classes are:

- From Class -1: Both $A(1, 2)$ and $B(2, 1)$.
- From Class +1: Both $C(5, 6)$ and $D(6, 5)$.

These are our **Support Vectors**. They lie exactly on the margin boundaries.

Step 2: Set up the Equations

For support vectors, the decision function equals exactly 1 or -1 :

- For Class +1 ($y_i=1$): $w \cdot x_i + b = 1$
- For Class -1 ($y_i=-1$): $w \cdot x_i + b = -1$

Let $w = [w_1, w_2]$. We can write the equations for our points:

1. For $C(5, 6)$: $5w_1 + 6w_2 + b = 1$
2. For $D(6, 5)$: $6w_1 + 5w_2 + b = 1$

3. For $A(1, 2)$: $1w_1 + 2w_2 + b = -1$

4. For $B(2, 1)$: $2w_1 + 1w_2 + b = -1$

Step 3: Solve the System

Find w_1 and w_2 : Subtract eq (2) from eq (1):

$$(5w_1 + 6w_2 + b) - (6w_1 + 5w_2 + b) = 1 - 1$$

$$-w_1 + w_2 = 0 \implies \mathbf{w_1 = w_2}$$

Let's call the weight w . Now substitute into (1) and (3):

- Eq (1): $5w + 6w + b = 1 \implies 11w + b = 1$

- Eq (3): $1w + 2w + b = -1 \implies 3w + b = -1$

Subtract the new Eq (3) from Eq (1):

$$(11w + b) - (3w + b) = 1 - (-1)$$

$$8w = 2 \implies \mathbf{w = 0.25}$$

So, $\mathbf{w = [0.25, 0.25]}$.

Find b : Substitute $w = 0.25$ back into $3w + b = -1$:

$$3(0.25) + b = -1$$

$$0.75 + b = -1 \implies \mathbf{b = -1.75}$$

Step 4: The Final Model

Decision Boundary: $0.25x_1 + 0.25x_2 - 1.75 = 0$

(Or multiplied by 4: $x_1 + x_2 = 7$)

Step 5: Verify the Margin

The margin width is $M = \frac{2}{\|w\|}$.

$$\|w\| = \sqrt{0.25^2 + 0.25^2} = \sqrt{0.0625 + 0.0625} = \sqrt{0.125} \approx 0.3536$$

$$M = \frac{2}{0.3536} \approx 5.66$$

Geometrically, distance between lines $x + y = 3$ and $x + y = 11$ is $4\sqrt{2} \approx 5.657$. Matches!

